

```

import React, { useState, useEffect, useRef } from 'react';
import { Upload, Plus, X, Camera, Pencil, Save, Sparkles } from 'lucide-react';

// ===== Storage helpers =====
const STORAGE_KEY = 'birthday-tribute-v3';

const loadData = async () => {
  try {
    const r = await window.storage.get(STORAGE_KEY);
    return r?.value ? JSON.parse(r.value) : null;
  } catch (e) {
    return null;
  }
};

const saveData = async (data) => {
  try {
    await window.storage.set(STORAGE_KEY, JSON.stringify(data));
  } catch (e) {
    console.error('Save failed:', e);
  }
};

// Resize images to keep storage under 5MB
const resizeImage = (file, maxW = 900) =>
  new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = (e) => {
      const img = new Image();
      img.onload = () => {
        const canvas = document.createElement('canvas');
        const scale = Math.min(1, maxW / img.width);
        canvas.width = img.width * scale;
        canvas.height = img.height * scale;
        const ctx = canvas.getContext('2d');
        ctx.drawImage(img, 0, 0, canvas.width, canvas.height);
        resolve(canvas.toDataURL('image/jpeg', 0.82));
      };
      img.onerror = reject;
      img.src = e.target.result;
    };
    reader.onerror = reject;
    reader.readAsDataURL(file);
  });

```

```
// ===== Default content =====
// ===== Embedded photos =====
const PHOTO_1 = "
const PHOTO_2 = "
const PHOTO_3 = "
const PHOTO_4 = "

const DEFAULT_DATA = {
  dadName: 'Xavier',
  age: '42',
  letterTitle: 'Gracias, papá',
  letter:
    'Gracias por todo el trabajo duro que haces. Gracias por levantarte temprano
  photos: [
    { id: 1, src: PHOTO_1, caption: 'Tú y yo, siempre' },
    { id: 2, src: PHOTO_2, caption: 'En las gradas, apoy\u00e9ndome' },
    { id: 3, src: PHOTO_3, caption: 'Tu pasi\u00f3n: Caramel' },
    { id: 4, src: PHOTO_4, caption: 'Domingo en familia' },
    { id: 5, src: null, caption: 'A\u00f1ade un recuerdo' },
    { id: 6, src: null, caption: 'A\u00f1ade un recuerdo' },
  ],
  messages: [
    {
      id: 1,
      from: 'Tu hijo Juan',
      text: 'Eres mi ejemplo, pap\u00e1. Todo lo que construyo es pensando en ti.
    },
    {
      id: 2,
      from: 'Kerly, tu esposa',
      text: 'Caminar contigo todos estos a\u00f1os ha sido el mejor regalo de mi
    },
    {
      id: 3,
      from: 'Tu familia',
      text: 'Eres el centro. El que re\u00fane, el que cuida, el que abraza fuert
    },
  ],
  wish: 'Que este nuevo a\u00f1o te traiga salud, risas largas, m\u00e1s tartas q
};

// ===== Confetti =====
const Confetti = () => {
  const pieces = Array.from({ length: 95 }, (_, i) => i);
  const colors = ['#E89B3A', '#B85A3F', '#B6743C', '#F5E8D0', '#D4A574', '#7A5230
  return (
```

```

<div className="fixed inset-0 pointer-events-none z-50 overflow-hidden">
  {pieces.map((i) => {
    const left = Math.random() * 100;
    const delay = Math.random() * 0.9;
    const duration = 2.8 + Math.random() * 2.5;
    const color = colors[Math.floor(Math.random() * colors.length)];
    const size = 7 + Math.random() * 13;
    const rotate = Math.random() * 360;
    const swing = (Math.random() - 0.5) * 140;
    const shape = i % 4;
    const sty = {
      position: 'absolute',
      left: `${left}%`,
      top: '-30px',
      animation: `confettiFall ${duration}s ${delay}s cubic-bezier(.25,.46,.4
        '--swing': `${swing}px`,
      transform: `rotate(${rotate}deg)`,
    };
    if (shape === 0) {
      return (
        <svg key={i} width={size} height={size} viewBox="0 0 24 24" fill={col
          <path d="M12 21s-7-4.4-9.3-9C7 8 3 4 7 4c2 0 3.5 1.2 5 3 1.5-1.8 3
        </svg>
      );
    }
    if (shape === 1) {
      return (
        <svg key={i} width={size} height={size} viewBox="0 0 24 24" fill={col
          <path d="M12 21s-7-4.4-9.3-9C7 8 3 4 7 4c2 0 3.5 1.2 5 3 1.5-1.8 3
        </svg>
      );
    }
    if (shape === 2) {
      return (
        <span key={i} style={{ ...sty, width: size * 0.7, height: size * 0.7,
        );
    }
    return (
      <span key={i} style={{ ...sty, width: size, height: size * 0.32, backgr
    );
  })}
</div>

);

};

// ===== Ambient sparkles (always on) =====
const AmbientSparkles = () => {

```

```

const items = Array.from({ length: 14 }, (_, i) => i);
return (
  <div className="fixed inset-0 pointer-events-none z-0 overflow-hidden">
    {items.map((i) => (
      <span
        key={i}
        style={{
          position: 'absolute',
          left: `${(i * 17 + 7) % 96}%`,
          top: `${(i * 29 + 11) % 92}%`,
          width: '3px',
          height: '3px',
          background: i % 3 === 0 ? '#E89B3A' : '#B6743C',
          borderRadius: '50%',
          opacity: 0,
          animation: `sparkleFloat ${5 + (i % 4)}s ${i * 0.5}s ease-in-out infi
          boxShadow: i % 3 === 0 ? '0 0 12px #E89B3A' : '0 0 8px #B6743C',
        }}
      />
    ))}
  </div>
);
};

// ===== Letter-by-letter reveal =====
const SplitText = ({ text, baseDelay = 0, charDelay = 0.07, className = '' }) =>
  const chars = String(text).split('');
  return (
    <span className={className} aria-label={text}>
      {chars.map((c, i) => (
        <span
          key={i}
          style={{
            display: 'inline-block',
            opacity: 0,
            animation: `letterDrop 0.75s ${baseDelay + i * charDelay}s cubic-bezi
          }}
        >
          {c === ' ' ? '\u00A0' : c}
        </span>
      ))}
    </span>
  );
};

// ===== Count-up number =====
const CountUp = ({ end, duration = 1800, startDelay = 0 }) => {

```

```

const [value, setValue] = useState(0);
const target = parseInt(end) || 0;
useEffect(() => {
  let raf;
  const startTime = performance.now() + startDelay;
  const step = (now) => {
    if (now < startTime) {
      raf = requestAnimationFrame(step);
      return;
    }
    const elapsed = now - startTime;
    const t = Math.min(1, elapsed / duration);
    const eased = 1 - Math.pow(1 - t, 3);
    setValue(Math.floor(eased * target));
    if (t < 1) raf = requestAnimationFrame(step);
    else setValue(target);
  };
  raf = requestAnimationFrame(step);
  return () => cancelAnimationFrame(raf);
}, [target, duration, startDelay]);
return <>{value}</>;
};

// ===== Candle =====
const Candle = ({ onLight }) => {
  const [lit, setLit] = useState(false);
  const handle = () => {
    if (!lit) {
      setLit(true);
      onLight();
    }
  };
  return (
    <button
      onClick={handle}
      aria-label="Encender la vela"
      className="relative w-9 h-16 group focus:outline-none"
    >
      {!lit && (
        <span
          aria-hidden
          style={{
            position: 'absolute',
            inset: '-12px',
            borderRadius: '50%',
            background: 'radial-gradient(circle, rgba(232,155,58,0.35) 0%, transp
            animation: 'candleAttention 2.2s ease-in-out infinite',

```

```

        pointerEvents: 'none',
    }}
/>
))
{/* Flame */}
<div
  className="absolute left-1/2 -translate-x-1/2 transition-all duration-500
  style={{
    bottom: '38px',
    opacity: lit ? 1 : 0.25,
    transform: `translateX(-50%) scale(${lit ? 1 : 0.6})`,
  }}
>
  <div
    className="w-3 h-5 rounded-full flame"
    style={{
      background: lit
        ? 'radial-gradient(ellipse at 50% 70%, #FFE082 0%, #FFB74D 40%, #E8
          : '#C9B9A0',
      boxShadow: lit
        ? '0 0 20px rgba(232,155,58,0.7), 0 0 40px rgba(232,155,58,0.4)'
        : 'none',
    }}
  />
</div>
{/* Wick */}
<div
  className="absolute left-1/2 -translate-x-1/2 w-[2px] h-2 bg-[#2D1810]"
  style={{ bottom: '34px' }}
/>
{/* Candle body */}
<div
  className="absolute left-1/2 -translate-x-1/2 bottom-0 w-3 h-9 rounded-sm
  style={{
    background:
      'linear-gradient(180deg, #F5E8D0 0%, #E8D5B0 50%, #C9A678 100%)',
    boxShadow: 'inset -2px 0 3px rgba(0,0,0,0.08)',
  }}
/>
{!lit && (
  <span className="absolute -bottom-5 left-1/2 -translate-x-1/2 text-[10px]
    tócame
  </span>
)}
</button>
);
};

```

```
// ===== Inline editable text =====

const EditableText = ({ value, onChange, multiline = false, className = '', place
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState(value);
  const ref = useRef(null);

  useEffect(() => setDraft(value), [value]);
  useEffect(() => {
    if (editing && ref.current) {
      ref.current.focus();
      if (!multiline) ref.current.select();
    }
  }, [editing, multiline]);

  const commit = () => {
    onChange(draft.trim() || placeholder || value);
    setEditing(false);
  };

  if (editing) {
    return multiline ? (
      <textarea
        ref={ref}
        value={draft}
        onChange={(e) => setDraft(e.target.value)}
        onBlur={commit}
        rows={4}
        className={` ${className} bg-[#FFF8E8] outline-none border-b-2 border-[#E8
      />
    ) : (
      <input
        ref={ref}
        value={draft}
        onChange={(e) => setDraft(e.target.value)}
        onBlur={commit}
        onKeyDown={(e) => e.key === 'Enter' && commit()}
        className={` ${className} bg-[#FFF8E8] outline-none border-b-2 border-[#E8
      />
    );
  }

  return (
    <span
      onClick={() => setEditing(true)}
      className={` ${className} cursor-text hover:bg-[#F5E8D0]/60 transition round
      title="Click para editar"
```

```

    >
    {value || <span className="opacity-40 italic">{placeholder}</span>}
  </span>
);
};

// ===== Photo slot =====
const PhotoSlot = ({ photo, onUpload, onRemove, onCaption, rotation }) => {
  const inputRef = useRef(null);

  const handleFile = async (e) => {
    const file = e.target.files?.[0];
    if (!file) return;
    try {
      const dataUrl = await resizeImage(file);
      onUpload(dataUrl);
    } catch (err) {
      alert('No se pudo cargar la imagen.');
```

```

    </>
  ) : (
    <button
      onClick={() => inputRef.current?.click()}
      className="absolute inset-0 flex flex-col items-center justify-center
    >
      <Camera size={28} strokeWidth={1.5} />
      <span className="text-xs font-body uppercase tracking-widest">
        subir foto
      </span>
    </button>
  )}
</div>
<div className="absolute bottom-3 left-3 right-3 text-center">
  <EditableText
    value={photo.caption}
    onChange={onCaption}
    className="font-script text-xl text-[#3A2418]"
    placeholder="añade una nota"
  />
</div>
</div>
);
};

// ===== Message card =====
const MessageCard = ({ msg, onUpdate, onDelete, rotation, tone }) => {
  const bg = tone === 'a' ? '#F5E8D0' : tone === 'b' ? '#FAE4C8' : '#F0DCB8';
  return (
    <div
      className="relative p-7 pb-8 shadow-[0_8px_24px_rgba(45,24,16,0.3)] t
      style={{ backgroundColor: bg, transform: `rotate(${rotation}deg)` }}
    >
      <button
        onClick={onDelete}
        className="absolute top-2 right-2 w-6 h-6 rounded-full bg-[#2D1810]/0 hov
        aria-label="Eliminar mensaje"
      >
        <X size={13} />
      </button>
      {/* tape effect */}
      <div className="absolute -top-3 left-1/2 -translate-x-1/2 w-14 h-5 bg-[#B67
      <p className="font-display italic text-lg leading-relaxed text-[#2D1810]">
        <EditableText
          value={msg.text}
          onChange={(v) => onUpdate({ ...msg, text: v })}
          multiline

```

```

        className="font-display italic"
      />
    </p>
    <p className="mt-4 font-script text-2xl text-[#B85A3F]">
      -{ ' ' }
      <EditableText
        value={msg.from}
        onChange={(v) => onUpdate({ ...msg, from: v })}
        className="font-script"
      />
    </p>
  </div>
);
};

// ===== Section reveal on scroll =====
const useReveal = () => {
  const ref = useRef(null);
  const [visible, setVisible] = useState(false);
  useEffect(() => {
    const obs = new IntersectionObserver(
      ([entry]) => entry.isIntersecting && setVisible(true),
      { threshold: 0.15 }
    );
    if (ref.current) obs.observe(ref.current);
    return () => obs.disconnect();
  }, []);
  return [ref, visible];
};

const Reveal = ({ children, delay = 0 }) => {
  const [ref, visible] = useReveal();
  return (
    <div
      ref={ref}
      style={{
        opacity: visible ? 1 : 0,
        transform: visible ? 'translateY(0) scale(1)' : 'translateY(40px) scale(0.85)',
        transition: `opacity 0.85s ${delay}s ease-out, transform 1s ${delay}s cubic-bezier(0.85, 0.15, 0.15, 0.85)`
      }}
    >
      {children}
    </div>
  );
};

// ===== Main app =====

```

```

export default function TributeSite() {
  const [data, setData] = useState(DEFAULT_DATA);
  const [loaded, setLoaded] = useState(false);
  const [showConfetti, setShowConfetti] = useState(false);
  const [savedFlash, setSavedFlash] = useState(false);
  const [introDone, setIntroDone] = useState(false);

  // Play intro animation once, then unlock normal editing
  useEffect(() => {
    const tIntro = setTimeout(() => setIntroDone(true), 3400);
    const tConfetti = setTimeout(() => {
      setShowConfetti(true);
      setTimeout(() => setShowConfetti(false), 4500);
    }, 1800);
    return () => {
      clearTimeout(tIntro);
      clearTimeout(tConfetti);
    };
  }, []);

  useEffect(() => {
    (async () => {
      const stored = await loadData();
      if (stored) setData(stored);
      setLoaded(true);
    })();
  }, []);

  useEffect(() => {
    if (!loaded) return;
    saveData(data);
    setSavedFlash(true);
    const t = setTimeout(() => setSavedFlash(false), 1500);
    return () => clearTimeout(t);
  }, [data, loaded]);

  const update = (field, value) => setData((d) => ({ ...d, [field]: value }));

  const updatePhoto = (id, patch) =>
    setData((d) => ({
      ...d,
      photos: d.photos.map((p) => (p.id === id ? { ...p, ...patch } : p)),
    }));

  const updateMessage = (msg) =>
    setData((d) => ({
      ...d,

```

```

      messages: d.messages.map((m) => (m.id === msg.id ? msg : m)),
    }));

const deleteMessage = (id) =>
  setData((d) => ({ ...d, messages: d.messages.filter((m) => m.id !== id) }));

const addMessage = () =>
  setData((d) => ({
    ...d,
    messages: [
      ...d.messages,
      { id: Date.now(), from: 'Alguien que te quiere', text: 'Escribe tu mensaj
    ],
  }));

const triggerCandle = () => {
  setShowConfetti(true);
  setTimeout(() => setShowConfetti(false), 4500);
};

// Pre-computed but stable rotations
const photoRotations = [-2.5, 1.8, -1.2, 2.3, -1.8, 1.4];
const msgRotations = [-1.4, 1.6, -0.8, 1.2, -1.0, 0.9];

return (
  <div className="min-h-screen w-full bg-[#FBF6EE] text-[#2D1810] font-body">
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Fraunces:ital,opsz,
      .font-display { font-family: 'Fraunces', Georgia, serif; font-optical-siz
      .font-script { font-family: 'Caveat', cursive; }
      .font-body { font-family: 'Inter', system-ui, -apple-system, sans-serif;

      @keyframes flicker {
        0%, 100% { opacity: 1; transform: translateY(0) scaleY(1); }
        50% { opacity: 0.88; transform: translateY(-1px) scaleY(0.95); }
      }
      .flame { animation: flicker 1.3s ease-in-out infinite; transform-origin:

      @keyframes confettiFall {
        0% { transform: translateY(0) rotate(0deg); opacity: 1; }
        100% { transform: translateY(110vh) rotate(720deg); opacity: 0; }
      }

      @keyframes confettiSway {
        0%, 100% { margin-left: 0; }
        50% { margin-left: var(--swing, 0px); }
      }
    `}
  </div>

```

```

@keyframes sparkleFloat {
  0%, 100% { opacity: 0; transform: translateY(0) scale(0.4); }
  50% { opacity: 0.85; transform: translateY(-28px) scale(1.1); }
}

@keyframes letterDrop {
  0% { opacity: 0; transform: translateY(-50px) rotate(-6deg); }
  60% { transform: translateY(6px) rotate(2deg); }
  100% { opacity: 1; transform: translateY(0) rotate(0); }
}

@keyframes candleAttention {
  0%, 100% { transform: scale(1); opacity: 0.55; }
  50% { transform: scale(1.4); opacity: 0.15; }
}

@keyframes nameGlow {
  0%, 100% { text-shadow: 0 0 20px rgba(232,155,58,0); }
  50% { text-shadow: 0 0 30px rgba(232,155,58,0.35), 0 0 60px rgba(184,90
}

.name-glow { animation: nameGlow 4s ease-in-out infinite; }

.paper {
  background-color: #FBF6EE;
  background-image:
    radial-gradient(circle at 20% 20%, rgba(184,90,63,0.05) 0%, transparen
    radial-gradient(circle at 80% 80%, rgba(232,155,58,0.06) 0%, transparen
    radial-gradient(circle at 60% 30%, rgba(122,82,48,0.03) 0%, transparen
}

@media (prefers-reduced-motion: reduce) {
  .flame { animation: none; }
  * { transition-duration: 0.01ms !important; animation-duration: 0.01ms
}
`</style>

<AmbientSparkles />
{showConfetti && <Confetti />}

{ /* Save indicator */ }
<div
  className="fixed top-4 right-4 z-40 flex items-center gap-2 text-xs font-
  style={{ opacity: savedFlash ? 1 : 0 }}
>
  <Save size={12} /> guardado
</div>

```

```

<div className="paper">
  {/* ===== HERO ===== */}
  <section className="relative min-h-screen flex flex-col justify-center px
    <div className="max-w-5xl mx-auto w-full">
      <Reveal>
        <div className="flex items-end gap-5 mb-2">
          <Candle onLight={triggerCandle} />
          <p className="font-script text-3xl sm:text-4xl text-[#B85A3F] pb-
            feliz cumpleaños
          </p>
        </div>
      </Reveal>

      <Reveal delay={0.15}>
        <h1 className="font-display font-medium italic text-[#2D1810] leadi
          <span className="block text-[18vw] sm:text-[15vw] md:text-[12vw]
            {introDone ? (
              <EditableText
                value={data.dadName}
                onChange={ (v) => update('dadName', v)}
              />
            ) : (
              <SplitText text={data.dadName} baseDelay={0.4} charDelay={0.0
            )}
          </span>
        </h1>
      </Reveal>

      <Reveal delay={0.3}>
        <div className="mt-10 flex items-baseline gap-4 sm:gap-6 flex-wrap"
          <span className="font-body text-[10px] sm:text-xs uppercase track
            cumple
          </span>
          <span className="font-display font-light text-[#B6743C] text-7xl
            {introDone ? (
              <EditableText value={data.age} onChange={ (v) => update('age',
            ) : (
              <CountUp end={data.age} startDelay={1400} duration={1600} />
            )}
          </span>
          <span className="font-body text-[10px] sm:text-xs uppercase track
            años
          </span>
        </div>
      </Reveal>

```

```

    <Reveal delay={0.45}>
      <div className="mt-16 flex items-center gap-3 text-[#7A5230]">
        <span className="h-px w-12 bg-[#B6743C]/50" />
        <span className="font-body text-[10px] uppercase tracking-[0.3em]
          desplaza para leer
        </span>
      </div>
    </Reveal>
  </div>
</section>

```

```

{ /* ===== LETTER ===== */}
<section className="px-6 sm:px-10 md:px-16 py-24 md:py-32">
  <div className="max-w-2xl mx-auto">
    <Reveal>
      <p className="font-body text-[10px] uppercase tracking-[0.35em] tex
        I · La carta
      </p>
    </Reveal>
    <Reveal delay={0.1}>
      <h2 className="font-display italic text-4xl sm:text-5xl text-[#2D18
        <EditableText
          value={data.letterTitle}
          onChange={(v) => update('letterTitle', v)}
        />
      </h2>
    </Reveal>
    <Reveal delay={0.2}>
      <div className="font-display text-lg sm:text-xl leading-relaxed tex
        <EditableText
          value={data.letter}
          onChange={(v) => update('letter', v)}
          multiline
          className="font-display"
        />
      </div>
    </Reveal>
  </div>
</section>

```

```

{ /* ===== PHOTO GALLERY ===== */}
<section className="px-6 sm:px-10 md:px-16 py-24 md:py-32 bg-[#F5E8D0]/40
  <div className="max-w-6xl mx-auto">
    <Reveal>
      <p className="font-body text-[10px] uppercase tracking-[0.35em] tex
        II · Momentos
      </p>
    </Reveal>
  </div>
</section>

```

```

    <h2 className="font-display italic text-4xl sm:text-5xl text-[#2D18
      Las fotos que te cuentan
    </h2>
    <p className="font-body text-sm text-[#7A5230] mb-12 max-w-md">
      Toca una para subir un recuerdo. Edita las notas al pie con un cl
    </p>
  </Reveal>

  <div className="grid grid-cols-2 md:grid-cols-3 gap-8 md:gap-10">
    {data.photos.map((p, i) => (
      <Reveal key={p.id} delay={i * 0.08}>
        <PhotoSlot
          photo={p}
          rotation={photoRotations[i % photoRotations.length]}
          onUpload={({src} => updatePhoto(p.id, { src })}
          onRemove={() => updatePhoto(p.id, { src: null })}
          onCaption={(caption) => updatePhoto(p.id, { caption })}
        </>
      </Reveal>
    ))}
  </div>
</div>
</section>

{ /* ===== MESSAGES ===== */ }
<section className="px-6 sm:px-10 md:px-16 py-24 md:py-32">
  <div className="max-w-5xl mx-auto">
    <Reveal>
      <p className="font-body text-[10px] uppercase tracking-[0.35em] tex
        III · Lo que queremos decirte
      </p>
      <h2 className="font-display italic text-4xl sm:text-5xl text-[#2D18
        Mensajes de los que te queremos
      </h2>
    </Reveal>

    <div className="grid sm:grid-cols-2 gap-8 md:gap-10">
      {data.messages.map((m, i) => (
        <Reveal key={m.id} delay={i * 0.1}>
          <MessageCard
            msg={m}
            rotation={msgRotations[i % msgRotations.length]}
            tone={['a', 'b', 'c'][i % 3]}
            onUpdate={updateMessage}
            onDelete={() => deleteMessage(m.id)}
          </>
        </Reveal>
      )}
    </div>
  </div>
</section>

```

```

    )))

    <Reveal delay={data.messages.length * 0.1}>
      <button
        onClick={addMessage}
        className="w-full min-h-[180px] border-2 border-dashed border-[
      >
        <Plus size={28} strokeWidth={1.5} className="group-hover:scale-
        <span className="font-body text-xs uppercase tracking-widest">
          añadir mensaje
        </span>
      </button>
    </Reveal>
  </div>
</div>
</section>

{ /* ===== WISH ===== */ }
<section className="px-6 sm:px-10 md:px-16 py-32 md:py-40">
  <div className="max-w-2xl mx-auto text-center">
    <Reveal>
      <Sparkles
        className="mx-auto mb-8 text-[#E89B3A]"
        size={32}
        strokeWidth={1.2}
      />
    </Reveal>
    <Reveal delay={0.15}>
      <p className="font-body text-[10px] uppercase tracking-[0.35em] tex
        IV · Nuestro deseo
      </p>
    </Reveal>
    <Reveal delay={0.3}>
      <p className="font-display italic text-2xl sm:text-3xl md:text-4xl
        <EditableText
          value={data.wish}
          onChange={ (v) => update('wish', v) }
          multiline
          className="font-display italic"
        />
      </p>
    </Reveal>
    <Reveal delay={0.5}>
      <div className="mt-16 font-script text-4xl text-[#B85A3F]">
        con todo nuestro amor
      </div>
    </Reveal>
  </div>
</section>

```

```
        </div>
    </section>

    { /* ===== Footer hint ===== */}
    <footer className="px-6 py-10 text-center">
        <p className="font-body text-[10px] uppercase tracking-[0.3em] text-[#B
            Hecho con cariño · {new Date().getFullYear()}
        </p>
    </footer>
</div>
</div>
);
}
```